

Java程序设计电子书

Java Programming from Basics to Advanced

作者：赵教授

学校编号：SC002 | 专业大类：软件

第一章 Java语言概述

1.1 Java的发展历程

Java语言由Sun

Microsystems公司于1995年正式发布，其设计目标是"一次编写，到处运行"（Write Once, Run Anywhere）。Java通过虚拟机（JVM）机制实现跨平台特性，程序编译后生成字节码（.class文件），可在任何安装了JVM的平台上运行。

Java版本演进：

-	Java 1.0 (1996)：	初始版本，提供基础语法和核心API	-	Java
5	(2004)：	引入泛型、增强for循环、注解等重大特性	-	Java
8	(2014)：	Lambda表达式、Stream API、新日期时间API	-	Java
11	(2018)：	LTS版本，引入var局部变量类型推断	-	Java
17	(2021)：	当前主流LTS版本，密封类、模式匹配等新特性		

1.2 Java开发环境搭建

JDK (Java Development Kit) 是Java开发的核心工具包，包含编译器 (javac)、运行时 (java)、调试器等工具。推荐安装OpenJDK 17 LTS版本。

环境变量配置： - JAVA_HOME：指向JDK安装目录 - PATH：添加%JAVA_HOME%\bin - CLASSPATH：指定类搜索路径

验证安装： java -version javac -version

第二章 面向对象编程基础

2.1 类与对象

类 (Class) 是对象的模板，定义了对象的属性和行为；对象 (Object) 是类的实例，具有具体的状态值。

```
public class Student { // 属性 (成员变量) private String name; private int age; private String studentId;
```

```
// 构造方法 public Student(String name, int age, String studentId) { this.name = name; this.age = age; this.studentId = studentId; }
```

```
// 方法 (行为) public void study() { System.out.println(name + "正在学习"); }
```

```
// Getter和Setter public String getName() { return name; } public void setName(String name) { this.name = name; } }
```

2.2 封装、继承、多态

封装：将数据和行为包装在类中，通过访问修饰符控制外部访问。private修饰的属性只能通过public方法访问，保证数据安全性。

继承：子类继承父类的属性和方法，实现代码复用。Java支持单继承，通过extends关键字实现。

```
public class Undergraduate extends Student { private String major;
```

```
public Undergraduate(String name, int age, String studentId, String major) { super(name, age, studentId); this.major = major; } }
```

多态：同一操作作用于不同对象产生不同行为。包括编译时多态（方法重载）和运行时多态（方法重写）。

第三章 集合框架

3.1 Collection接口体系

Java集合框架提供了一套标准化的数据结构和算法实现，位于java.util包中。

主要接口：

- List：有序集合，允许重复元素。实现类：ArrayList、LinkedList、Vector
- Set：无序集合，不允许重复元素。实现类：HashSet、TreeSet、LinkedHashSet
- Queue：队列，FIFO原则。实现类：LinkedList、PriorityQueue、ArrayDeque

3.2 Map接口体系

Map存储键值对（key-value），键唯一，值可重复。

主要实现类：

- HashMap：基于哈希表，查询效率 $O(1)$ ，无序
- TreeMap：基于红黑树，按键排序，查询效率 $O(\log n)$
- LinkedHashMap：保持插入顺序
- ConcurrentHashMap：线程安全，高并发场景首选

使用示例：

```
Map scores = new HashMap<>(); scores.put("张三", 90); scores.put("李四", 85); scores.put("王五", 92);
```

```
for (Map.Entry entry : scores.entrySet()) { System.out.println(entry.getKey() + ": " + entry.getValue()); }
```

第四章 异常处理与I/O流

4.1 异常处理机制

Java将异常分为受检异常（Checked Exception）和非受检异常（Unchecked Exception）。受检异常必须在方法签名中声明或捕获处理，非受检异常包括RuntimeException及其子类。

```
try { FileReader reader = new FileReader("data.txt"); // 读取文件内容 } catch
(FileNotFoundException e) { System.err.println("文件不存在：" + e.getMessage()); } catch
(IOException e) { System.err.println("读取错误：" + e.getMessage()); } finally { // 关闭资源 }
```

4.2 输入输出流

Java I/O流分为字节流（InputStream/OutputStream）和字符流（Reader/Writer）。字节流处理二进制数据，字符流处理文本数据并自动进行编码转换。

常用流类：

- FileInputStream/FileOutputStream：文件字节流
- FileReader/FileWriter：文件字符流
- BufferedReader/BufferedWriter：缓冲流，提高读写效率
- ObjectInputStream/ObjectOutputStream：对象序列化流

第五章 多线程编程

5.1 线程创建方式

方式一：继承Thread类 `class MyThread extends Thread { @Override public void run() { System.out.println("线程执行"); }}`

方式二：实现Runnable接口（推荐） `class MyTask implements Runnable { @Override public void run() { System.out.println("任务执行"); }}`

方式三：实现Callable接口（可返回结果） `class MyCallable implements Callable { @Override public String call() throws Exception { return "任务结果"; }}`

5.2 线程同步

当多个线程访问共享资源时，需要使用同步机制保证数据一致性。

synchronized关键字：

- 同步方法：`public synchronized void method() {}`
- 同步块：`synchronized (obj) { ... }`

```
Lock接口（更灵活的锁机制）： ReentrantLock lock = new ReentrantLock(); lock.lock(); try {  
// 临界区代码 } finally { lock.unlock(); }
```

本书配套所有章节的示例代码和课后习题，读者可访问出版社网站下载完整源码。建议每章学习后完成相应编程练习，巩固所学知识。